

Computer Science as a Layered Discipline

Foundations, Systems, Intelligence, and Societal Responsibility

Review Article and Conceptual Synthesis

SAMUELSON G

Independent Researcher

Email: gsamuelsonguna@gmail.com

ORCID: 0009-0005-8744-8178

13 July 2026

Research integrity note: This article presents a literature-based synthesis and an original conceptual framework. It does not report new experiments, human-subject research, animal research, or fabricated measurements.

Abstract

Computer science is simultaneously a mathematical science of computation, an engineering discipline for building reliable information-processing systems, and a sociotechnical practice whose artifacts shape institutions and everyday life. This review develops a layered account of the field that links mathematical foundations, representations and algorithms, computer systems, software and data abstractions, intelligent and interactive applications, and societal governance. The article uses a targeted narrative review of seminal research, current curricular frameworks, and authoritative standards to identify durable concepts and emerging priorities. Four claims organize the synthesis. First, abstraction is the field's central intellectual mechanism: each layer hides lower-level detail while exporting guarantees, costs, and failure modes. Second, computer science research is evaluated through plural forms of evidence, including proof, asymptotic analysis, simulation, controlled experiment, benchmark comparison, systems measurement, user study, and field observation. Third, modern systems cannot be evaluated by accuracy or speed alone; security, privacy, reliability, maintainability, energy use, accessibility, fairness, and human control are cross-cutting quality attributes. Fourth, the most consequential research frontiers arise at interfaces among subfields, including trustworthy artificial intelligence, quantum-safe security, edge and cyber-physical systems, sustainable computing, formal verification, and human-centered interaction. The paper contributes an original layered model of computer science, a reusable research-to-deployment lifecycle, and a research agenda centered on compositional assurance, lifecycle evaluation, resource-aware intelligence, secure migration, and public-interest computing. The synthesis is intended as a reference for researchers, educators, students, and practitioners who need a coherent map of computer science without reducing it to programming or to any single technological wave.

Keywords: computer science; algorithms; computer systems; software engineering; artificial intelligence; cybersecurity; human-computer interaction; computing ethics

1. Introduction

Computer science studies computation: what can be computed, how efficiently it can be computed, how computational processes can be represented and controlled, and how computing systems interact with people and the physical world. Programming is an indispensable practice within the discipline, but it is not an adequate definition of the discipline. Computer science includes mathematical models, hardware-software interfaces, communication protocols, data organization, intelligent behavior, human interaction, verification, security, and the institutional consequences of deploying computational systems.

The field's modern foundations emerged from several distinct intellectual traditions. Turing formalized effective computation and limits of decidability [1]. Shannon developed a

quantitative theory of information and communication [2]. Later work transformed programming from an ad hoc craft into a discipline of structured control and formal reasoning [3], [4]; established data independence through the relational model [5]; explained ordering in distributed systems [6]; and created public-key cryptography [7]. These contributions did not merely add topics. They established reusable abstractions that made new layers of systems possible.

Computer science has since expanded into a family of interdependent subfields. The ACM Computing Classification System provides a broad research taxonomy [10], while the ACM/IEEE Computing Curricula 2020 report and the ACM/IEEE-CS/AAAI Computer Science Curricula 2023 report describe computing as a set of related disciplines and competencies rather than a single skill set [11], [12]. This expansion creates a communication problem: specialists can optimize one layer while overlooking constraints or harms that appear elsewhere.

This article addresses that problem through a review and conceptual synthesis. Its central question is: What coherent structure connects the theoretical, engineered, empirical, and societal dimensions of computer science? The answer proposed here is a layered model in which foundations support algorithms; algorithms are realized by systems; systems expose software and data abstractions; applications create intelligent and interactive capabilities; and all layers operate within human, organizational, legal, and ecological contexts. Security, trust, efficiency, and human factors cut across every layer.

2. Review Scope and Method

This paper is a targeted narrative review rather than a systematic review or meta-analysis. Sources were selected to represent three categories: (1) seminal works that established durable concepts; (2) contemporary curricular and classification frameworks that map the field; and (3) authoritative standards and recent research that illuminate current frontiers. The review prioritizes primary research articles, professional-society publications, and standards from ACM, IEEE, NIST, and established scholarly venues. Searches and source verification were completed through July 2026.

The synthesis process had four stages. First, concepts were grouped by the level of abstraction at which they operate. Second, each group was characterized by its central questions, artifacts, evidence, and characteristic failure modes. Third, cross-cutting qualities were traced across levels. Fourth, emerging frontiers were assessed according to technical opportunity, dependency on other subfields, evaluation difficulty, and societal risk. Because the topic is the whole discipline, coverage is necessarily selective. The goal is a coherent map, not an exhaustive bibliography.

3. Intellectual Foundations

3.1 Computability, complexity, and algorithms

Computability theory separates problems that admit effective procedures from those that do not. Turing's machine model remains important because it abstracts away physical implementation and focuses on symbolic steps [1]. Complexity theory refines this question by asking how resource requirements grow with input size. Algorithm analysis uses upper and lower bounds to compare procedures independently of a particular machine, while practical algorithm engineering adds constants, memory behavior, parallelism, numerical stability, and workload structure.

$$T(n) = O(f(n)) \text{ when there exist } c > 0 \text{ and } n^0 \text{ such that } T(n) \leq c f(n) \text{ for all } n \geq n^0 \quad (1)$$

Asymptotic notation does not predict wall-clock time by itself, but it identifies growth regimes and scalability limits. A theoretically superior method may lose on small inputs, while an apparently fast implementation may collapse at scale. Sound evaluation therefore combines mathematical analysis with measurements on representative data and hardware.

Table 1. Representative growth rates and their practical interpretation.

Growth	Typical example	Interpretation	Common caution
$O(1)$	Array index access	Cost does not grow with input length under the model	Memory hierarchy can dominate actual latency
$O(\log n)$	Balanced-tree search	Problem size can multiply with modest additional work	Requires balance and suitable comparison model
$O(n)$	Full scan	Work grows proportionally with data	Bandwidth and cache behavior matter
$O(n \log n)$	Comparison sorting	Common near-optimal regime for ordering	Input distribution and parallelism affect constants
$O(n^2)$	All-pairs comparison	Often acceptable only for modest n	Vectorization can delay, not remove, scaling limits
$O(2^n)$ or $O(n!)$	Exact combinatorial search	Rapidly becomes infeasible	Approximation, heuristics, parameterization, or structure are needed

3.2 Information, representation, and uncertainty

Information theory provides a quantitative language for uncertainty, coding, and communication. For a discrete random variable X with outcomes x and probabilities $p(x)$, Shannon entropy is:

$$H(X) = -\sum_x p(x) \log_2 p(x) \quad (2)$$

The equation links representation to probability: common events can be encoded compactly, while noise and channel constraints limit reliable transmission [2]. In computer science, representation is never neutral. Data types, schemas, coordinate systems, tokenization rules, numerical precision, and feature choices determine which operations are efficient and which distinctions can be expressed. Many system failures are therefore representation failures before they are algorithm failures.

3.3 Abstraction, correctness, and modularity

Abstraction manages complexity by exposing selected properties while hiding implementation detail. Structured programming reduced ungoverned control flow [3]. Hoare logic expressed program behavior through preconditions, commands, and postconditions [4]. Codd's relational model separated logical data relationships from physical storage [5]. In distributed computing, Lamport showed that causality and event ordering require explicit logical structure when no global clock exists [6]. Each case converts an implementation problem into an interface with reasoned guarantees.

Modularity is valuable only when contracts are explicit. Hidden coupling reappears through timing, shared state, data distributions, dependency versions, or organizational procedures. Brooks' distinction between essential and accidental complexity remains relevant: tools can remove some implementation burden, but they do not eliminate the difficulty of specifying, coordinating, and validating complex behavior [8].

4. A Layered Model of Computer Science

Figure 1 presents the paper's organizing framework. The layers are not a rigid hierarchy. Research frequently moves downward, as when an AI workload motivates a new accelerator, or sideways, as when cryptography changes a networking protocol. Nevertheless, the model clarifies where claims originate and where obligations propagate.

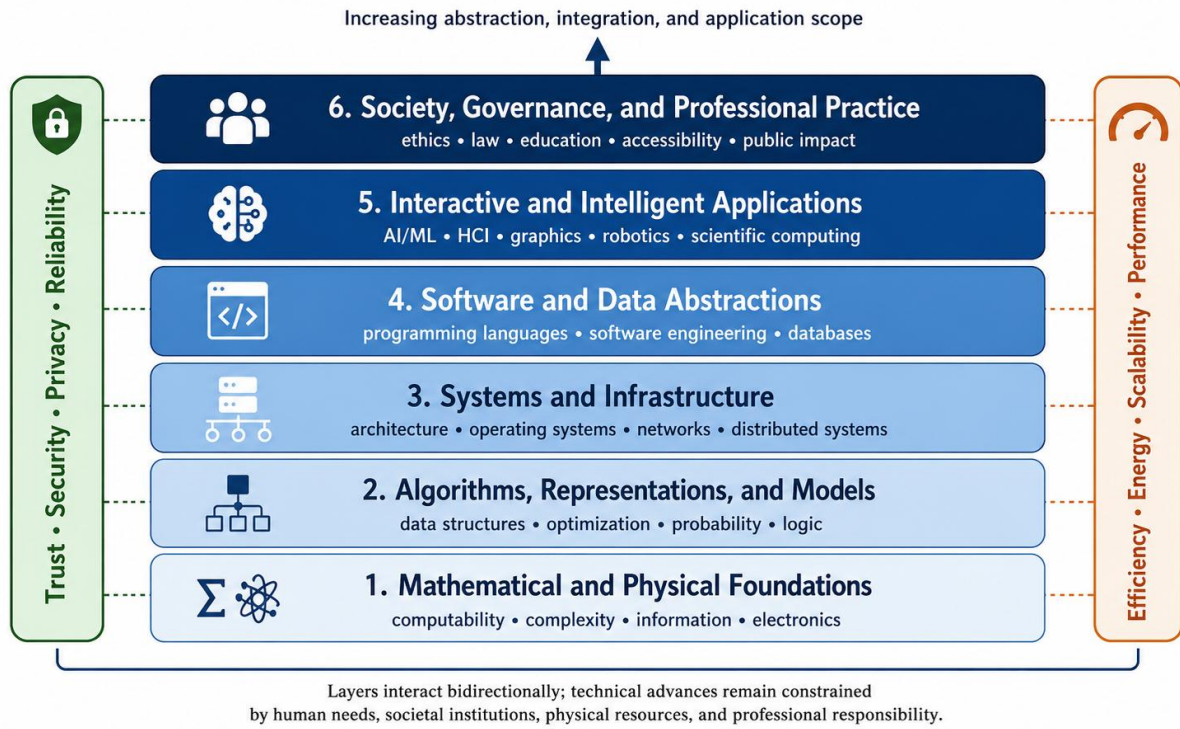


Figure 1. Layered model of computer science. The diagram is an original synthesis by the author.

At the foundation are computability, complexity, probability, logic, and physical information processing. Above them are representations, algorithms, and mathematical models. Systems layers realize those ideas in architectures, operating systems, networks, storage, and distributed infrastructure. Software and data abstractions make infrastructure programmable and maintainable. Intelligent and interactive applications create user-facing behavior. The outer layer includes institutions, professional duties, regulation, education, labor, culture, and environmental constraints. Trust and efficiency are depicted as vertical pillars because a security flaw, energy bottleneck, inaccessible interface, or reliability defect can invalidate the whole stack.

Table 2. Major domains of computer science within the layered model.

Domain	Central question	Typical artifact	Characteristic evidence
Theory and algorithms	What is computable, and at what resource cost?	Proof, algorithm, reduction, approximation	Correctness proof, complexity bound, competitive analysis
Architecture and embedded systems	How should computation be physically organized?	Processor, accelerator, memory hierarchy, device	Throughput, latency, energy, area, fault behavior

Domain	Central question	Typical artifact	Characteristic evidence
Operating systems and networks	How are shared resources coordinated and communicated?	Kernel, protocol, scheduler, network service	Trace analysis, benchmark, stress test, failure injection
Programming languages and software engineering	How can complex behavior be expressed, verified, evolved, and maintained?	Language, compiler, library, software architecture	Type soundness, proof, testing, defect data, maintainability study
Data management	How can data be modeled, queried, governed, and recovered?	Schema, database engine, stream processor	Correctness, consistency, query performance, recovery tests
Artificial intelligence and machine learning	How can systems infer, learn, reason, generate, or act under uncertainty?	Model, agent, training pipeline, benchmark	Generalization, robustness, calibration, human evaluation
HCI, graphics, and visualization	How do people perceive, control, understand, and collaborate with systems?	Interface, visualization, interaction technique	User study, accessibility audit, task performance, field study
Security and privacy	How are assets protected under adversarial conditions?	Protocol, control, proof, detection system	Threat model, reduction, penetration test, incident analysis
Computational science and robotics	How can computation model, control, and discover in physical domains?	Simulator, controller, scientific workflow	Validation against observation, sensitivity analysis, reproducibility

5. Core Domains and Their Interdependence

5.1 Computer architecture and specialized hardware

Computer architecture mediates between abstract programs and physical devices. Classical performance improvements relied on faster transistors, instruction-level parallelism, caches, and multicore designs. More recent architecture research emphasizes domain-specific acceleration, open instruction sets, security, memory systems, and hardware-software co-design [13]. The central lesson is that performance is workload-dependent. A processor optimized for dense linear algebra, encrypted communication, or real-time control may perform poorly outside its design envelope.

Amdahl's law illustrates why optimizing only one component has diminishing returns. If fraction p of a workload can be accelerated by N while the remainder is unchanged, the maximum speedup is:

$$S(N) = 1 / [(1 - p) + p/N] \quad (3)$$

The equation generalizes beyond parallel computing: system-level benefit is limited by unoptimized stages, data movement, synchronization, and operational overhead. Energy and data movement increasingly belong in the same objective function as speed.

5.2 Operating systems, networking, and distributed systems

Operating systems provide isolation, scheduling, memory, storage, and device abstractions. Networks extend resource sharing across administrative and geographic boundaries. Distributed systems then confront partial failure, concurrency, variable delay, replication, and the absence of a universal notion of time. Lamport's logical clocks made causality explicit [6], while Internet protocol design demonstrated how layered end-to-end communication can scale across heterogeneous networks [31].

Large-scale data processing frameworks such as MapReduce simplified distributed execution by constraining the programming model and automating partitioning, scheduling, and recovery [14]. The CAP discussion later clarified that consistency and availability trade-offs become unavoidable under network partitions, but practical systems offer a spectrum of guarantees rather than a simplistic two-of-three choice [15]. The enduring research challenge is compositional reasoning: local component guarantees do not automatically become end-to-end service guarantees.

5.3 Programming languages and software engineering

Programming languages determine which invariants can be expressed and checked. Type systems, ownership models, contracts, formal semantics, and automated verification shift errors from deployment time toward design or compilation time. Software engineering adds architecture, requirements, testing, version control, review, observability, maintenance, and organizational coordination. The product is not source code alone; it is a maintained capability embedded in people, dependencies, data, and operations.

Modern software systems depend on deep supply chains. Package ecosystems and automated deployment improve reuse but expand the attack surface and complicate provenance. The relevant unit of correctness is therefore often a lifecycle rather than a program. Long-lived systems require migration strategies, deprecation policies, reproducible builds, incident response, and explicit ownership.

5.4 Data management and knowledge infrastructure

Codd's relational model established data independence and declarative querying [5]. Contemporary systems extend these ideas to distributed databases, streams, graphs, vector indexes, and data lakes. The core concerns remain representation, integrity, concurrency, query planning, lineage, and recovery. Data quality and governance are inseparable from algorithmic quality: a model or decision service inherits the omissions, biases, leakage, and drift of its data pipeline.

A useful distinction is between data availability and epistemic adequacy. A large dataset may still be unrepresentative, ambiguously labeled, or collected for a different purpose. Computer science research must document provenance, transformations, access conditions, and intended use, not only storage size or benchmark accuracy.

5.5 Artificial intelligence and machine learning

Machine learning replaces some hand-specified rules with models estimated from data. Deep learning demonstrated the value of multilayer representation learning across perception and prediction [16], and the Transformer architecture enabled highly parallel sequence modeling based on attention [17]. Foundation models extend this approach by training broad models that can be adapted to many downstream tasks, creating leverage as well as correlated failure and governance risk [18].

A common formalization is empirical risk minimization, where model parameters are chosen to minimize loss over observed examples:

$$\theta^* = \arg \min_{\theta} [(1/n) \sum_i L(f_{\theta}(x_i), y_i) + \lambda R(\theta)] \quad (4)$$

Equation 4 exposes only part of the engineering system. Real performance also depends on data collection, preprocessing, feedback, serving infrastructure, monitoring, access control, and human oversight. Hidden technical debt can accumulate through unstable dependencies, entangled features, configuration, and undeclared consumers [19]. Evaluation must therefore include robustness, calibration, uncertainty, distribution shift, misuse resistance, and post-deployment behavior, not only average benchmark scores.

5.6 Human-computer interaction, graphics, and visualization

Human-computer interaction treats people as active participants rather than external recipients. Interfaces shape what users notice, understand, trust, and can correct. Research methods include controlled experiments, qualitative inquiry, accessibility evaluation, participatory design, and field observation. Graphics and visualization add models of perception, geometry, rendering, interaction, and explanatory communication.

Usability is not cosmetic. A system can be computationally correct yet operationally unsafe if it hides uncertainty, overloads attention, excludes users with disabilities, or makes recovery difficult. Human control should therefore be designed as a system property with clear feedback, reversible actions, appropriate automation, and escalation paths.

5.7 Cybersecurity and privacy

Security research assumes intelligent adversaries and asks what assets, capabilities, and trust boundaries define the threat model. Diffie and Hellman transformed key establishment and authentication by introducing public-key concepts [7]. Saltzer and Schroeder articulated design principles such as least privilege, fail-safe defaults, complete mediation, and open design [20]. These principles remain relevant because security failures often arise from architecture and incentives rather than from a single defective cipher.

Quantum computing creates a migration problem for widely deployed public-key systems. NIST finalized FIPS 203, FIPS 204, and FIPS 205 in 2024 for quantum-resistant key encapsulation and digital signatures [24]-[26]. The computer science challenge is not merely selecting algorithms; it includes inventorying cryptographic dependencies, designing hybrid transitions, updating protocols and hardware, preserving interoperability, and managing long-lived data whose confidentiality must survive future attacks.

6. Research Methods and Standards of Evidence

Computer science is methodologically plural. A theorem may be the strongest possible evidence for an abstract claim, while it may say little about constant factors or human use. A benchmark can reveal comparative performance, but it may reward specialization to a narrow dataset. A user study can reveal interaction effects, but it may not predict long-term institutional outcomes. Rigorous work aligns the claim, method, baseline, metric, and validity argument.

Table 3. Common research methods and validity concerns in computer science.

Method	Best suited claim	Strength	Frequent validity threat
Formal proof	Correctness, impossibility, bound, security reduction	General statement under explicit assumptions	Assumptions omit implementation or environment
Asymptotic analysis	Scalability and resource growth	Machine-independent comparison	Constants and data movement are hidden
Simulation	Behavior of models or	Repeatability and	Model mismatch with

Method	Best suited claim	Strength	Frequent validity threat
	systems under controlled conditions	parameter control	reality
Prototype and benchmark	Performance or feasibility of an implementation	Concrete evidence on real hardware	Benchmark leakage, tuning bias, weak baselines
Randomized controlled experiment	Causal effect under controlled treatment	Strong internal validity	Artificial tasks or short duration
User study and qualitative inquiry	Usability, comprehension, practice, lived experience	Reveals human mechanisms and context	Small or unrepresentative samples
Field deployment or case study	Operational impact and sociotechnical behavior	High ecological validity	Confounding and limited generalizability
Adversarial evaluation	Robustness and security under attack	Tests worst-case behavior	Incomplete attacker model or test coverage
Replication and artifact evaluation	Reproducibility and portability	Detects hidden dependencies	Artifacts may not capture production context

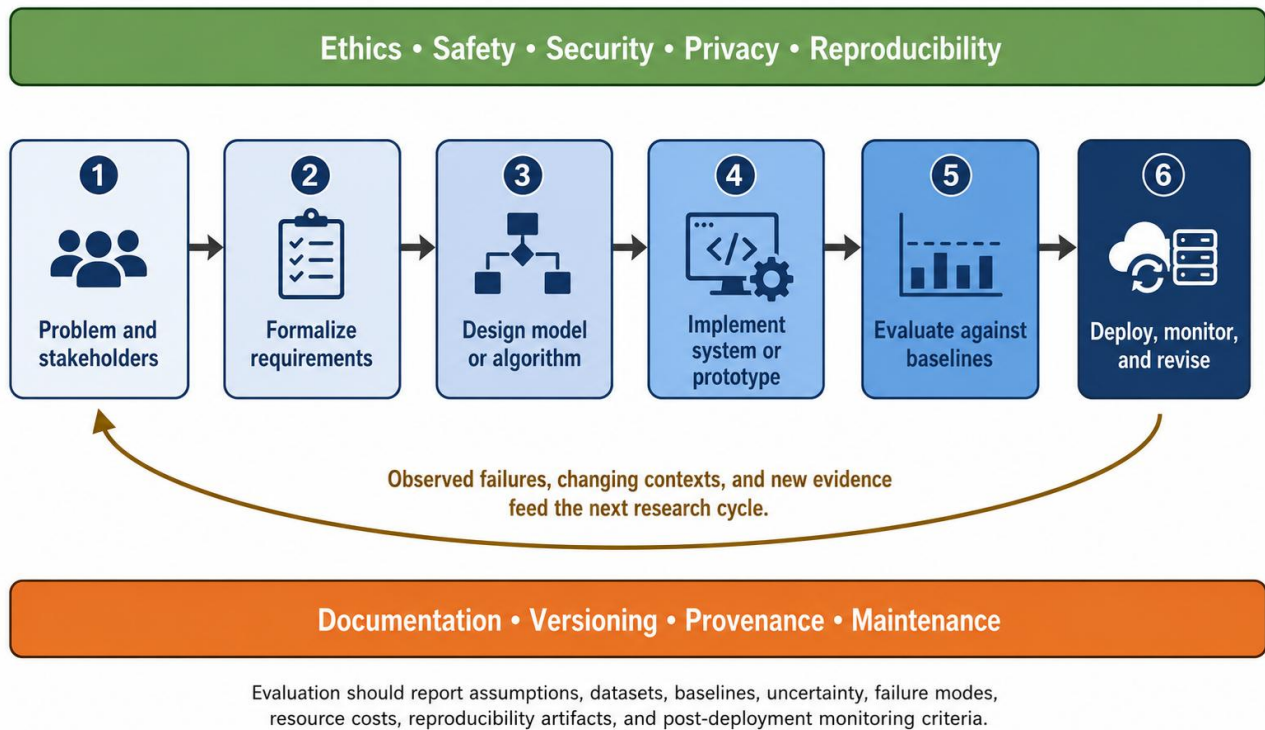


Figure 2. Research-to-deployment lifecycle for computer science. The model is an original synthesis by the author.

Figure 2 makes post-deployment monitoring part of research validity. Systems alter the environment that produces future data, and users adapt to systems. A one-time evaluation can therefore become obsolete. Mature computer science needs versioned datasets, model and software cards, reproducible environments, telemetry with privacy controls, incident reporting, and criteria for rollback or retirement.

7. Cross-Cutting Design Tensions

7.1 Performance versus generality

Generality promotes reuse and portability, while specialization can improve speed and energy efficiency. Domain-specific accelerators, specialized databases, and task-specific models exploit structure, but they increase fragmentation and migration cost. The research objective is not maximum generality or maximum specialization; it is an explicit boundary with measurable benefits and escape paths.

7.2 Consistency versus availability and latency

Distributed systems must choose how much coordination to impose. Stronger consistency simplifies reasoning but can increase latency and reduce availability under faults. Weaker

consistency improves responsiveness but transfers complexity to applications and users. Correct design requires matching the guarantee to the harm of stale, duplicated, reordered, or conflicting information.

7.3 Accuracy versus interpretability and control

Predictive accuracy is valuable only relative to a decision context. In high-impact settings, uncertainty, explanation, contestability, provenance, and human authority can be as important as a small improvement in average accuracy. NIST's AI Risk Management Framework organizes trustworthy AI work around governance, mapping, measurement, and management [22], with a generative AI profile adding risks specific to broad generative systems [23].

7.4 Security versus usability and openness

Security controls impose cognitive and operational costs. Poorly designed controls are bypassed, shared, or disabled. Conversely, frictionless systems can externalize risk to users or society. Security must be usable, minimally privileged, observable, and recoverable. Open design supports scrutiny, but openness does not remove the need to protect secrets, personal data, and abuse-sensitive operational detail.

7.5 Scale versus sustainability

Scaling computation can increase capability but also energy use, water consumption, hardware demand, and electronic waste. Efficiency gains may be consumed by larger workloads. Sustainable computing therefore requires lifecycle accounting, workload-aware design, hardware longevity, carbon-aware scheduling, and reporting that distinguishes training, inference, data movement, cooling, and embodied impacts. Energy and policy concerns associated with large-scale learning have already become a recognized research issue [28].

8. Emerging Frontiers

Table 4. Selected research frontiers, opportunities, and assurance needs.

Frontier	Technical opportunity	Primary risk	High-priority evidence
Trustworthy and agentic AI	Systems that plan, use tools, and adapt across tasks	Unreliable autonomy, misuse, correlated downstream failure	Capability boundaries, red teaming, calibrated uncertainty, human-

Frontier	Technical opportunity	Primary risk	High-priority evidence
			control tests
Quantum computing and quantum-safe migration	New algorithms for simulation, optimization, and cryptography	Overclaiming near-term advantage; insecure transition	Resource estimates, error analysis, reproducible comparisons, migration trials
Edge and cyber-physical systems	Low-latency local intelligence integrated with sensing and control	Safety failures, physical attack, limited updateability	Real-time guarantees, fault injection, safety cases, hardware-in-the-loop tests
Sustainable computing	Higher useful work per unit of energy, material, and carbon	Burden shifting and rebound effects	Lifecycle metrics, workload transparency, sensitivity analysis
Formal verification at scale	Machine-checked assurance for software, hardware, protocols, and AI components	Specification error and proof-maintenance cost	Verified toolchains, compositional proofs, independent checking
Human-centered and accessible computing	Systems aligned with diverse abilities, values, and contexts	Exclusion, manipulation, overreliance	Participatory design, accessibility audits, longitudinal field evidence

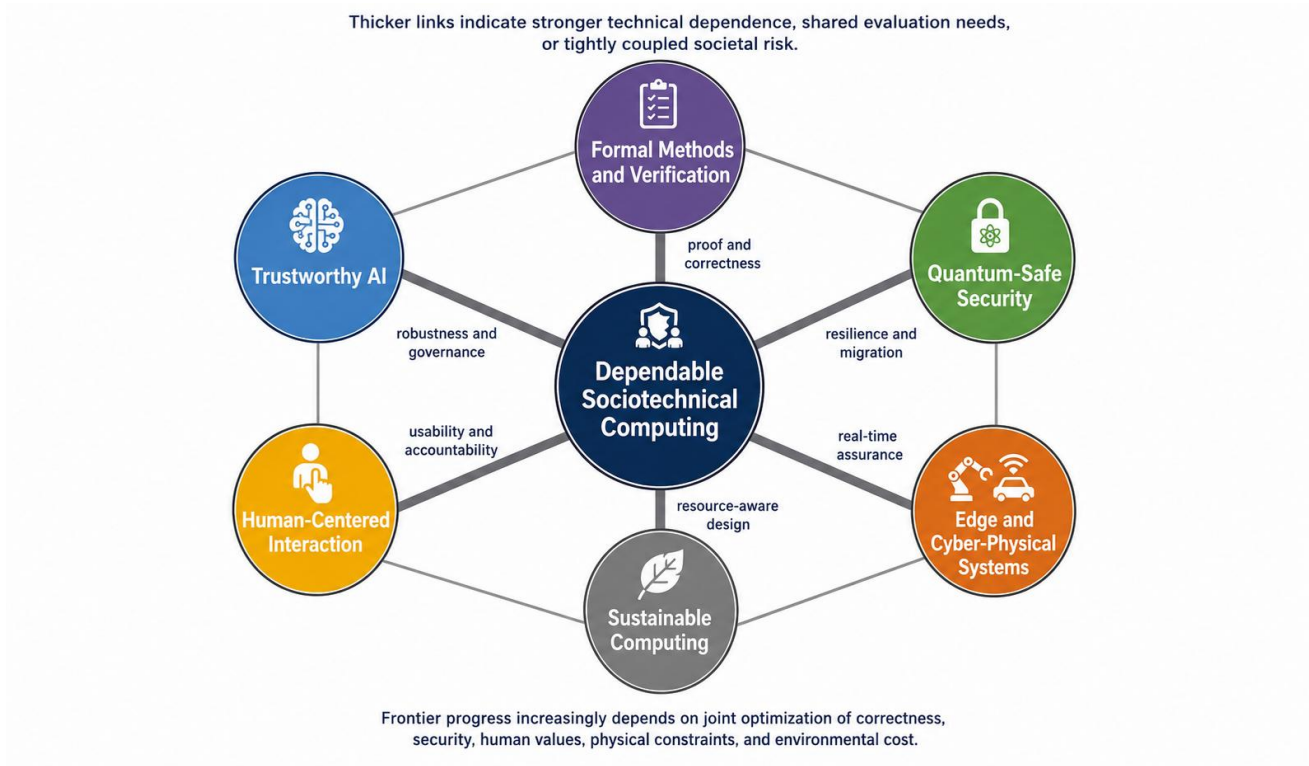


Figure 3. Convergence among selected research frontiers. The network is an original conceptual map by the author.

Figure 3 emphasizes convergence. Trustworthy AI depends on human-computer interaction, secure infrastructure, formal assurance, and sustainable resource use. Quantum-safe security depends on algorithms, protocols, hardware, software supply chains, and organizational migration. Edge intelligence couples real-time systems with privacy, safety, energy, and physical robustness. The highest-value research may therefore be interface research that makes guarantees compose across technical and institutional boundaries.

Quantum computing illustrates both opportunity and epistemic caution. Feynman argued that quantum systems motivate new forms of computation for simulating physics [27]. Yet practical advantage depends on error rates, algorithmic structure, hardware scale, compilation, verification, and fair comparison with improving classical methods. Similar caution applies to AI: impressive capability does not eliminate the need for clear task definitions, baselines, failure analysis, and governance.

9. Research Agenda

9.1 Compositional assurance

Develop methods that combine proofs, tests, runtime monitors, and human procedures into end-to-end assurance cases. Research should state which assumptions are machine-checked, empirically tested, operationally monitored, or delegated to institutions.

9.2 Lifecycle evaluation

Replace one-time benchmark thinking with longitudinal evaluation of drift, maintenance, incidents, user adaptation, energy use, and decommissioning. Public artifacts should include versioned data, configurations, dependency manifests, and known failure envelopes.

9.3 Resource-aware intelligence

Treat compute, memory, communication, energy, latency, and human attention as first-class resources in AI and systems research. Report capability per resource, not capability alone.

9.4 Secure and agile migration

Study how large software and cryptographic ecosystems can change without synchronized global replacement. Hybrid protocols, compatibility layers, inventories, automated dependency discovery, and staged retirement require both technical and organizational research.

9.5 Human agency and public-interest computing

Design systems that make uncertainty visible, support appeal and correction, protect vulnerable users, and align incentives with the public good. The ACM Code of Ethics places the public good at the center of professional responsibility [21].

9.6 Education for integration

Curricula should preserve mathematical depth while teaching systems, data, AI, security, ethics, and communication as connected competencies. CS2023's competency orientation supports this integration [12]. Students should learn to trace a claim from formal model to implementation, measurement, user impact, and governance.

10. Limitations

A review of an entire discipline must omit important areas and debates. This paper does not provide a bibliometric analysis, exhaustive taxonomy, or quantitative comparison of subfield growth. It emphasizes durable concepts and cross-cutting issues, which may underrepresent highly specialized research. The frontier assessment is interpretive rather than

predictive. In addition, professional and regulatory requirements vary by jurisdiction and application domain. Readers preparing a submission should adapt terminology, declarations, and references to the target venue.

11. Conclusion

Computer science is best understood not as programming alone and not as a loose list of technologies, but as a layered discipline organized by abstraction. Mathematical foundations define possibilities and limits. Algorithms and representations transform those foundations into procedures. Architectures, operating systems, networks, and distributed systems make procedures executable at scale. Programming languages, software engineering, and data systems make them reusable and maintainable. Artificial intelligence, visualization, robotics, and interactive systems convert them into capabilities. Society supplies purposes, constraints, rights, risks, and accountability.

The field's future depends on integrating evidence across these layers. Proof without implementation can miss operational reality; benchmarks without theory can reward brittle optimization; products without human and societal evaluation can scale harm. The most important research frontiers are consequently connective: trustworthy AI, quantum-safe infrastructure, verified systems, sustainable computing, accessible interaction, and lifecycle assurance. A mature computer science asks not only whether a system can be built, but what it guarantees, what it costs, how it fails, who can control it, and whether its benefits justify its risks.

Declarations

Author contribution: SAMUELSON G is listed as the sole author and is responsible for reviewing, revising, approving, and taking accountability for the final submitted manuscript.

Funding: No external funding is received for this review manuscript.

Competing interests: The author declares no known financial or non-financial competing interests.

Ethics approval: Not applicable. This article does not involve human participants, identifiable personal data, animals, or clinical intervention.

Consent to participate and publish: Not applicable.

Data and materials availability: No new empirical dataset was created. The article is based on publicly available scholarly publications and standards cited in the reference list. Original conceptual figures are included in the manuscript.

Code availability: No research software or executable analysis code is required to reproduce the literature synthesis. Figure-generation source code should be archived with the final repository if the manuscript is publicly released.

References

- [1] A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society*, s2-42(1), pp. 230-265, 1936, doi: 10.1112/plms/s2-42.1.230.
- [2] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379-423 and 623-656, 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [3] E. W. Dijkstra, "Letters to the editor: Go to statement considered harmful," *Communications of the ACM*, vol. 11, no. 3, pp. 147-148, 1968, doi: 10.1145/362929.362947.
- [4] C. A. R. Hoare, "An axiomatic basis for computer programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576-580, 1969, doi: 10.1145/363235.363259.
- [5] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377-387, 1970, doi: 10.1145/362384.362685.
- [6] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558-565, 1978, doi: 10.1145/359545.359563.
- [7] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, 1976, doi: 10.1109/TIT.1976.1055638.
- [8] F. P. Brooks, Jr., "No silver bullet: Essence and accidents of software engineering," *Computer*, vol. 20, no. 4, pp. 10-19, 1987, doi: 10.1109/MC.1987.1663532.
- [9] P. J. Denning et al., "Computing as a discipline," *Communications of the ACM*, vol. 32, no. 1, pp. 9-23, 1989, doi: 10.1145/63238.63239.
- [10] Association for Computing Machinery, "ACM Computing Classification System," 2012. Available through the ACM Digital Library.
- [11] ACM and IEEE Computer Society, *Computing Curricula 2020: Paradigms for Global Computing Education*. New York, NY: ACM, 2020, doi: 10.1145/3467967.
- [12] ACM, IEEE Computer Society, and AAI, *Computer Science Curricula 2023*. New York, NY: ACM, 2024, doi: 10.1145/3664191.
- [13] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48-60, 2019, doi: 10.1145/3282307.
- [14] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107-113, 2008, doi: 10.1145/1327452.1327492.
- [15] E. A. Brewer, "CAP twelve years later: How the rules have changed," *Computer*, vol. 45, no. 2, pp. 23-29, 2012, doi: 10.1109/MC.2012.37.

- [16] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436-444, 2015, doi: 10.1038/nature14539.
- [17] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems* 30, pp. 5998-6008, 2017.
- [18] R. Bommasani et al., "On the opportunities and risks of foundation models," *arXiv:2108.07258*, 2021.
- [19] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems* 28, pp. 2503-2511, 2015.
- [20] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278-1308, 1975, doi: 10.1109/PROC.1975.9939.
- [21] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*. New York, NY: ACM, 2018.
- [22] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023, doi: 10.6028/NIST.AI.100-1.
- [23] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, 2024, doi: 10.6028/NIST.AI.600-1.
- [24] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, 2024, doi: 10.6028/NIST.FIPS.203.
- [25] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, 2024, doi: 10.6028/NIST.FIPS.204.
- [26] National Institute of Standards and Technology, *Stateless Hash-Based Digital Signature Standard*, FIPS 205, 2024, doi: 10.6028/NIST.FIPS.205.
- [27] R. P. Feynman, "Simulating physics with computers," *International Journal of Theoretical Physics*, vol. 21, pp. 467-488, 1982, doi: 10.1007/BF02650179.
- [28] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645-3650, 2019, doi: 10.18653/v1/P19-1355.
- [29] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1983.
- [30] B. Shneiderman, *Human-Centered AI*. Oxford, UK: Oxford University Press, 2022.
- [31] V. G. Cerf and R. E. Kahn, "A protocol for packet network intercommunication," *IEEE Transactions on Communications*, vol. 22, no. 5, pp. 637-648, 1974, doi: 10.1109/TCOM.1974.1092259.
- [32] B. Rous, "Major update to ACM's Computing Classification System," *Communications of the ACM*, vol. 55, no. 11, pp. 12-13, 2012, doi: 10.1145/2366316.2366320.